Final Report

Assignment 1

Pham Minh Hieu

September 2025

The concept and knowledge are the main idea to form my algorithm but I still need to use Cursor to fix bugs when it comes to numerical instability (NaN, big different with answers or type mismatching). Big thanks to the lecturer, TA, classmates and internet resources.

1 Problem 1

- Approach: The first problem strictly demonstrates the class content about FlashAttention. Reviewing the presentation and checking for Pytorch syntax is enough to perform the whole algorithm

- Key learning:

- Tiling the attention matrix into blocks for efficient memory computation
- Using PyTorch to implement forward pass with causal masking ($causal\_mask = k\_ind < q\_ind$)
- Applying online softmax for faster computation O(n)

2 Problem 2

- Approach: The second problem is simply the introduction to triton user. Hints are very helpful to match with the Triton documents.

- Key learning:

- Triton syntax (load, dot, where, maximum, exp and exp2)
- Block-based processing and pointer arithmetic (remind me of cpp)
- Masking for out-of-bounds access ($mask = col\_offsets < N\_COLS$)

3 Problem 3

- Approach: This is an implementation of simple FlashAttention to Triton (without masking).

- Key learning:

- Triton syntax (load, dot, where, maximum, exp and exp2)
- Emphasizing SRAM accumulators, online softmax updates, and block-wise dot products
- Adjusting between dtype for numerical stability

4 Problem 4

- Approach: Adds causal masking to the kernel with a two-phase approach. Follow the instruction to access K and V blocks. Finish by the same online softmax code from problem 3

- Key learning:

- Handling of triangular matrices and partial blocks
- Understanding the need of logic and flexibility for two-phase program

5 Problem 5

- Approach: Review Multi-Query Attention, Grouped-Query Attention papers and Youtube videos (I recommend DataMListic) for deeper knowledge and clean visualization.

- Key learning:

- Mapping multiple query heads to shared key/value heads ($heads\_per\_group = N\_Q\_HEADS//N\_KV\_HEADS$)
- Head grouping logic for scalable attention ($kv\_head\_idx = q\_head\_idx//heads\_per\_group$)

6 Problem 6

- Approach: Review papers, FlashAttention code (from TriDao github) and Youtube videos (DataMListic) to integrate Sliding Window Attention for long-sequence efficiency.

- Key learning:

- Using aligned window starts to limit attention scope ($window\_start = tl.maximum(0, q\_block\_idx * BLOCK\_M - WINDOW\_SIZE)$)
- Head Per-element masking to avoid non-value key during online softmax ($row\_oke = row\_max > -float('inf')$)

7 Problem 7

- Approach: Review papers, FlashAttention code (from TriDao github), Youtube videos (DataMListic) and discuss with my classmates (mostly I ask them) to learn about Attention Sinks.

- Key learning:

- Stabilizing SWA by always attending to initial tokens
- Combined masking with three phase kernel (sink, off-diagonal, diagonal) to prevent divergence in long sequences.

8 Problem 8

- Approach: the goal was to implement the backward pass for FlashAttention-2 with GQA support, enabling fine-tuning of models by computing gradients for queries (dq), keys (dk), and values (dv) efficiently. I started by extending the forward pass from previous problems to save intermediate tensors (q, k, v, o, and log-sum-exp M) in the autograd context. The forward kernel used a two-phase structure (off-diagonal

and diagonal blocks) for causal attention, with online softmax for numerical stability, and GQA mapping to share key/value heads. For backward pass, I follow the instructions: set grid and blocks, recompute attention scores and probabilities, compute Delta and dS_ij, accumulate gradients. Due to precision and stability errors, I have to perform all computations in tl.float32

- Key learning:

- Understand the workflow of backpropagation in fine-tuning LLMs.
- Using tl.atomic_add for shared key/value heads to handle concurrency in Triton kernels, ensuring correct gradient summation.
- Triton and GQA reduces memory by sharing keys/values, but complicates gradient accumulation, highlighting the balance between memory savings and computational overhead.

9 Problem 9

- Approach: I believe Problem 9 extends Problem 8 to implement the backward pass for FlashAttention with GQA, Sliding Window Attention (SWA), and Attention Sinks. I reused the forward kernel from Problem 7, which includes three phases (sink, off-diagonal, diagonal) with combined masking for causal, window, and sink constraints, saving M (log-sum-exp) for recomputation. For the backward kernel, the strategy shows similarity with Problem 8 but different in masking techinque ($combined\_mask = causal\_mask \& (swa\_mask | sink\_mask)$)

- Key learning:

- Clearly understand the whole workflow of fine-tuning LLMs with FlashAttention, but I'm sure the problem is much more complex when it come to full fine-tuning, not just through testing with naive approach.
- Combining causal, SWA, and sink masks in recomputation taught me how to maintain exact gradients for constrained attention, ensuring no leakage from invalid positions while supporting features like long-context stability in LLMs.
- Learning how sink masks stabilizes training for streaming inputs by forcing attention to initial tokens, preventing model collapse in long dialogues which is a practical lesson from GPT-OSS's adoption.